APEX FX TRADING BOT

Master AI Coding Prompt

Complete Implementation Guide — PRD Volume I + Volume II

What this is	A ready-to-paste system prompt for your AI coding assistant containing every specification, audit finding, and instruction needed to complete APEX FX correctly and completely.
Existing code	Partially built — see Section 2 for the full audit of what works and what needs fixing before Volume II work begins.
PRD references	Volume I v1.0 + Volume II v2.0. Both are authoritative. Volume II extends Volume I — neither replaces the other.
How to use	Give your AI the entire prompt in Section 3 onwards — verbatim. Do not shorten it. Then in each follow-up message, ask it to implement one module at a time in the order specified in Section 3.7.

This document was produced after a full cross-reference of the Technical Documentation v1.0 against PRD Volumes I and II. Every gap, inconsistency, and missing component has been identified and documented. Your AI's job is to resolve them all.

Section 1 — Your Guide (Human Reading — Not Part of the Prompt)

This section is for you — the developer — not for your AI assistant. Read it once, then skip to Section 3 when giving instructions to your AI.

1.1 What the Existing Code Already Has (Do Not Rebuild)

These components are working per the Technical Documentation v1.0. Instruct your AI to extend these files, never rewrite them from scratch.

- MT5Connector — full MT5 connection, OHLC data, order open/close/get
- EconomicCalendar — news event fetch, blackout checks per symbol, gold-specific 45-min blackout
- CommodityFeed — WTI crude price, trending direction for USD/CAD correlation
- RegimeDetector — TRENDING / RANGING / BREAKOUT_PENDING / AVOID classification
- All 6 strategy classes — per-instrument signal logic with indicators
- RiskManager — ATR position sizing, stop/TP calculation, circuit breakers
- CorrelationManager — EUR+CHF, GBP+EUR, CAD+Gold cross-pair exposure limits
- OrderRouter — order placement with spread check and 3-attempt retry
- TelegramAlert — trade alerts, daily report, heartbeat
- Flask API — all existing endpoints in src/api/main.py
- SQLite database — 5 tables via database.py
- Web dashboard — templates/index.html

1.2 What Needs to Be Built (11 New Modules — Volume II)

These do not exist. They must be created as new files in the correct src/ subfolder.

- src/risk/fill_analyzer.py — post-fill execution quality, Broker Quality Score
- src/risk/signal_scorer.py — weighted 0-100 signal scoring system
- src/risk/kelly_sizer.py — fractional Kelly adaptive position sizing
- src/risk/volatility_scaler.py — VRS portfolio vol scalar, XAU Defensive Mode
- src/risk/portfolio_heat_monitor.py — real-time aggregate open risk tracking
- src/risk/regime_drift_detector.py — weekly strategy health vs backtest baseline
- src/analysis/performance_analyzer.py — Friday self-learning feedback loop
- src/data/cot_parser.py — CFTC COT report fetcher and COT Index calculator
- src/execution/time_exit_manager.py — max hold time, forward progress, weekend policy
- src/execution/calendar_filter.py — day-of-week, month-end, quarter-end rules
- src/data/swap_filter.py — rollover cost check before entries near 22:00 UTC

1.3 What Needs to Be Modified (6 Existing Files Extended)

- src/risk/manager.py — add cold streak ladder, Vol II drawdown modes, heat monitor calls
- src/execution/order_router.py — extend to 4-attempt retry with exact ms delays; add order splitting
- src/data/mt5_connector.py — add symbol normalization map, swap rate fetch

- src/monitoring/telegram_alerts.py — add 4-tier alert system (INFO/WARNING/CRITICAL/EMERGENCY)
- src/api/main.py — add API endpoints for all 11 new modules
- main.py — wire all new modules into startup and main execution loop

1.4 Recommended Session Order

Ask your AI to implement one module per session in this order. Starting with risk modules ensures the foundation is solid before adding intelligence layers:

- Session 1: Fix the 8 critical gaps listed in Section 2 of this document
- Session 2: portfolio_heat_monitor.py + volatility_scaler.py
- Session 3: time_exit_manager.py + calendar_filter.py + swap_filter.py
- Session 4: signal_scorer.py + fill_analyzer.py
- Session 5: cot_parser.py + wire COT into signal_scorer
- Session 6: kelly_sizer.py + update manager.py integration
- Session 7: regime_drift_detector.py + performance_analyzer.py
- Session 8: Update all API endpoints + Telegram alert tiers + main.py wiring
- Session 9: Full integration test — run all modules together on demo account

Section 2 — Full Code Audit & Critical Gaps

This section documents all discrepancies found between the Technical Documentation v1.0 and the PRD Volume I + II specifications. These must be fixed before or alongside the Volume II additions.

2.1 Critical Gaps — Fix These First

Priority 1: These are Volume I requirements missing from the existing code. They represent actual trading risk. Fix all 8 before building any Volume II module.

Critical Gap 1 — Trailing Stops Not Applied in the Main Loop

Status: METHODS EXIST BUT NOT CALLED. RiskManager has should_activate_trailing_stop() and calculate_trailing_stop() but the main execution loop does not iterate open positions and call these on every H1 bar close.

Required fix: In the main scan loop (or a separate monitoring loop), after every strategy scan, fetch all open positions via mt5_connector.get_positions(). For each position, call should_activate_trailing_stop(). If True, compute new trailing SL and call order_router.modify_position(ticket, sl=new_sl). Log every trailing stop adjustment to trades.log.

Critical Gap 2 — Symbol Name Inconsistency (USDCAD vs USD/CAD)

Status: POTENTIAL RUNTIME ERROR. The STRATEGIES dict uses 'USDCAD' but MT5 brokers may use 'USDCAD', 'USD/CAD', or 'USDCAD.raw' depending on the account. Without normalization this will cause mt5.symbol_info() to return None and silently skip the pair.

Required fix: Add SYMBOL_MAP dict to MT5Connector mapping canonical names to broker-specific names. Load from mt5_config.yaml. Apply normalization in get_ohlc() and open_order() before any MT5 call. Log a warning at startup if any watchlist symbol is not found in MT5.

Critical Gap 3 — Position Sizing Formula Mismatch

Status: FORMULA DISCREPANCY. Technical docs show: Lot = (Balance × Risk%) / (SL_Pips × ATR_Mult × Pip_Value). PRD Volume I Section 5.2 formula is: Lot = (Balance × Risk%) / (ATR_Multiplier × ATR(14) × Pip_Value). The stop distance in the PRD IS the ATR × multiplier — there is no separate SL_Pips variable. The denominator must be: ATR14_value × atr_multiplier × pip_value_per_lot.

Required fix: Audit calculate_position_size() in manager.py line by line. The SL distance passed to it must be the raw ATR × multiplier distance in price units, not pip count. Confirm pip_value is fetched dynamically from MT5 symbol_info().tick_value, never hardcoded.

Critical Gap 4 — No Spread-to-ATR Ratio Filter

Status: MISSING ENTIRELY. PRD Volume II Section 21.1 mandates: block entry if current_spread / ATR(14)_H1 > 0.15. This is not in order_router.py.

Required fix: Add check_spread_atr_ratio(symbol, atr14_h1) method to OrderRouter. Fetch current spread as (ask - bid) / point. Divide by ATR14 value in price units. If ratio > 0.15, return (False, 'Spread-ATR ratio {ratio:.3f} exceeds 0.15 limit'). Call this in place_order() before the existing static spread check.

Critical Gap 5 — Per-Pair Daily Loss Tracking Incomplete

Status: LIMITS DEFINED BUT ENFORCEMENT UNCLEAR. Volume I Section 5.3 specifies per-pair max daily loss: EUR/USD 3%, GBP/USD 3%, USD/JPY 3%, USD/CHF 2.5%, USD/CAD 3%, XAU/USD 2%. The

check_trade_allowed() method references this but the underlying daily P&L query by symbol is not clearly implemented.

Required fix: Add get_daily_realised_pnl(symbol) to database.py — sum all closed trades for symbol since UTC midnight. Add get_daily_unrealised_pnl(symbol) from open positions. In check_trade_allowed(), compute total_daily_loss_pct = (realised + unrealised loss) / account_balance × 100. Block if this exceeds the instrument's max daily loss from INSTRUMENT_PROFILES.

Critical Gap 6 — Retry Logic: 3 Attempts Instead of Required 4

Status: PARTIALLY COMPLIANT. order_router.py has max 3 retries. Volume II Section 21.3 requires exactly 4 attempts: immediate / 500ms / 1500ms / 4000ms with slippage tolerance escalating at 0.3 / 0.5 / 0.8 / 1.2 pips. On 4th failure, log as MISSED_SIGNAL, send WARNING alert, and do NOT open the trade.

Required fix: Rewrite the retry loop to exactly 4 attempts. Pass increasing deviation parameter to mt5.order_send() per attempt. After final failure, write a MISSED_SIGNAL record to the signals table and trigger a WARNING tier Telegram alert.

Critical Gap 7 — No Time-Based Trade Exits

Status: MISSING ENTIRELY. No maximum hold time exists. Trades can remain open indefinitely, accumulating overnight risk and tying up capital in failed signals.

Required fix: Build time_exit_manager.py (Section 3.5 of this prompt). Wire into the main monitoring loop. On every H1 bar, check every open position's open time against the max hold time table. Apply forward progress rule. On Friday after 20:00 UTC, apply weekend position policy.

Critical Gap 8 — No Weekend Position Policy

Status: MISSING ENTIRELY. Per Volume II Section 19.3: after 16:00 UTC Friday no new entries. At 20:00 UTC Friday, close all positions in loss; tighten trailing stops on profitable positions; only hold through weekend if profit >= 1.0x ATR and trailing stop is active.

Required fix: Include weekend policy in time_exit_manager.py. Add a friday_close_check() method called on every bar when datetime.weekday() == 4 (Friday) and hour >= 20.

2.2 Minor Gaps — Fix Alongside Volume II Work

- USD/CAD EIA Wednesday filter: Verify calendar_feed.py blocks trades 30 min before/after 15:30 UTC on Wednesdays. If not present, add it.
- BoJ Intervention persistence: Store the intervention flag and its expiry timestamp in the SQLite settings table so it survives bot restarts.
- OPEC event filter for USD/CAD: Add OPEC meeting detection to calendar_feed.py. During OPEC meetings, reduce USD/CAD position size by 50% via a size_modifier returned alongside the blackout flag.
- APScheduler jobs missing: Add scheduled jobs in main.py: daily report at 22:00 UTC, RDD run at 21:00 UTC Friday, PerformanceAnalyzer at 21:30 UTC Friday, COT parser update at 16:30 UTC Friday.
- Monthly drawdown window: Confirm it uses a rolling 30-day window from now backward, not a calendar month reset. Fix if it resets on the 1st of each month.
- Database.py migration: Add the new tables required by Volume II modules (see Section 3.1 of this prompt for the complete schema additions).

Section 3 — Master Coding Prompt (Paste This to Your AI)

PASTE EVERYTHING BELOW THIS BOX TO YOUR AI ASSISTANT VERBATIM

Then ask it to implement one module per session in the order shown in Section 3.7

PART 1 — YOUR ROLE

You are a senior quantitative systems engineer completing and upgrading the APEX FX Trading Bot — a fully automated Python algorithmic trading system for MetaTrader 5. The bot trades EUR/USD, GBP/USD, USD/JPY, USD/CHF, USD/CAD, and XAU/USD (Gold).

The codebase is partially built. You have two responsibilities: (1) fix specific gaps and bugs in the existing code, and (2) build 11 new professional-grade modules specified in PRD Volume II. Do not rewrite working components. Extend them.

Before writing any code, read this entire prompt. The modules are interconnected. Understanding the full system first prevents architectural mistakes that require rewrites later.

PART 2 — MANDATORY CODING RULES

Apply these rules to every file without exception:

- Python 3.11+ with full type hints on every function signature.
- Every function has a docstring explaining its purpose, parameters, return value, and any exceptions it may raise.
- All exceptions are caught, logged to errors.log via the existing structlog logger, and re-raised or handled gracefully — the bot must never crash silently.
- All configurable values live in YAML or JSON config files. Nothing is hardcoded. Add new config keys to the appropriate existing config file or create a new vol2_config.yaml.
- All new database operations use the existing SQLiteManager in src/data/database.py. Add new tables as migration methods on the DatabaseManager class.
- Shared state (heat monitor values, daily P&L accumulators, Kelly trade history, RDD status) must be protected with threading.Lock() — the Flask API and strategy scanner run on separate threads.
- All datetimes are timezone-aware UTC: use datetime.now(timezone.utc), never datetime.utcnow() (deprecated in Python 3.12).
- Symbol names: use the SYMBOL_MAP in MT5Connector to translate between canonical names (e.g. 'EURUSD') and broker-specific names before every MT5 API call.
- Never use print() statements. Use the existing logger from src/monitoring/logger.py for all output.
- After writing each new module, write a corresponding test file in tests/ using pytest. Each test must mock MT5 connections and database calls — tests must run without a live MT5 terminal.

PART 3 — DATABASE SCHEMA ADDITIONS

Add the following new tables to database.py as migration methods. Call all migrations in DatabaseManager.__init__() so they run automatically on startup. Use IF NOT EXISTS on all CREATE TABLE statements.

New Table: fill_quality

```
CREATE TABLE IF NOT EXISTS fill_quality (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    symbol TEXT NOT NULL,
    ticket INTEGER,
    expected_price REAL,
    actual_fill_price REAL,
    execution_shortfall_pips REAL,
    spread_at_submission REAL,
    spread_at_fill REAL,
    fill_latency_ms REAL,
    was_requoted INTEGER DEFAULT 0,
    was_partial_fill INTEGER DEFAULT 0
);
```

New Table: signal_scores

```
CREATE TABLE IF NOT EXISTS signal_scores (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    symbol TEXT NOT NULL,
    direction TEXT NOT NULL,
    trend_alignment_score REAL,
    momentum_score REAL,
    regime_quality_score REAL,
    session_timing_score REAL,
    execution_quality_score REAL,
    cot_bonus REAL DEFAULT 0,
    total_score REAL NOT NULL,
    grade TEXT NOT NULL,
    position_modifier REAL NOT NULL,
    trade_opened INTEGER DEFAULT 0,
    outcome_pips REAL
);
```

New Table: kelly_history

```
CREATE TABLE IF NOT EXISTS kelly_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    calculated_at TEXT NOT NULL,
    symbol TEXT NOT NULL,
    lookback_trades INTEGER,
    win_rate REAL,
    avg_win_r REAL,
    avg_loss_r REAL,
    full_kelly REAL,
    fractional_kelly REAL,
    effective_risk_pct REAL
);
```

New Table: portfolio_heat_log

```
CREATE TABLE IF NOT EXISTS portfolio_heat_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    heat_pct REAL NOT NULL,
    heat_level TEXT NOT NULL,
    open_positions_count INTEGER,
```

```
    action_taken TEXT
);
```

New Table: rdd_status

```
CREATE TABLE IF NOT EXISTS rdd_status (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    checked_at TEXT NOT NULL,
    symbol TEXT NOT NULL,
    status TEXT NOT NULL,
    live_profit_factor REAL,
    baseline_profit_factor REAL,
    performance_ratio REAL,
    action_taken TEXT
);
```

New Table: cot_data

```
CREATE TABLE IF NOT EXISTS cot_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    report_date TEXT NOT NULL,
    symbol TEXT NOT NULL,
    net_speculative_position REAL,
    cot_index REAL,
    direction TEXT,
    is_extreme INTEGER DEFAULT 0
);
```

New Table: weekly_reports

```
CREATE TABLE IF NOT EXISTS weekly_reports (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    week_ending TEXT NOT NULL,
    net_pnl REAL,
    total_trades INTEGER,
    win_rate REAL,
    profit_factor REAL,
    max_drawdown_week REAL,
    peak_portfolio_heat REAL,
    avg_bqs REAL,
    report_json TEXT
);
```

PART 4 — CRITICAL BUG FIXES (DO THESE FIRST)

Implement all 8 fixes below before creating any new Volume II module. These fix Volume I compliance gaps in the existing code.

Fix 1 — Wire Trailing Stops into Main Loop

In the main execution loop, after strategy scanning, add a trailing stop monitoring block:

```
def monitor_trailing_stops(self):
    """Check and update trailing stops for all open positions on every bar."""
```

```
    positions = self.mt5.get_positions()
    for pos in positions:
        symbol = pos['symbol']
        direction = pos['type']  # 'BUY' or 'SELL'
        entry = pos['open_price']
        current_price = pos['current_price']
        profit_pips = pos['profit_pips']
        current_sl = pos['sl']
        ticket = pos['ticket']
        should_trail = self.risk_manager.should_activate_trailing_stop(
            symbol, current_price, entry, direction, profit_pips
        )
        if should_trail:
            new_sl = self.risk_manager.calculate_trailing_stop(
                symbol, current_price, entry, direction
            )
            # Only update if new SL is better than current SL
            if direction == 'BUY' and new_sl > current_sl:
                self.order_router.modify_position(ticket, sl=new_sl)
            elif direction == 'SELL' and new_sl < current_sl:
                self.order_router.modify_position(ticket, sl=new_sl)
```

Fix 2 — Symbol Normalization in MT5Connector

Add to mt5_connector.py:

```
# Add to MT5Connector.__init__():
self.symbol_map: dict[str, str] = config.get('symbol_map', {
    'EURUSD': 'EURUSD', 'GBPUSD': 'GBPUSD', 'USDJPY': 'USDJPY',
    'USDCHF': 'USDCHF', 'USDCAD': 'USDCAD', 'XAUUSD': 'XAUUSD'
})

def normalize_symbol(self, symbol: str) -> str:
    """Translate canonical symbol name to broker-specific name."""
    return self.symbol_map.get(symbol, symbol)

def validate_symbols(self) -> list[str]:
    """At startup, verify all watchlist symbols exist in MT5. Log warnings for
missing ones."""
    missing = []
    for canonical, broker_name in self.symbol_map.items():
        info = mt5.symbol_info(broker_name)
        if info is None:
            self.logger.warning('symbol_not_found', canonical=canonical,
broker_name=broker_name)
            missing.append(canonical)
    return missing
```

Fix 3 — Position Sizing Formula Correction

Audit calculate_position_size() in manager.py. The correct PRD formula is:

```
def calculate_position_size(self, symbol: str, account_balance: float,
                            atr14: float, direction: str) -> float:
    """
    PRD Volume I Section 5.2 formula:
    Lot Size = (Account Balance * Risk%) / (ATR_Multiplier * ATR14 *
Pip_Value_per_lot)
    where atr14 is the raw ATR value in price units (e.g. 0.00085 for EURUSD)
    """
    profile = self.INSTRUMENT_PROFILES[symbol]
```

```
    risk_amount = account_balance * (profile['risk_per_trade'] / 100)
    atr_stop_distance = profile['atr_multiplier'] * atr14  # in price units
    # pip_value = MT5 tick_value * (atr_stop_distance / tick_size)
    symbol_info = mt5.symbol_info(self.mt5.normalize_symbol(symbol))
    tick_value = symbol_info.trade_tick_value
    tick_size = symbol_info.trade_tick_size
    pip_value_per_lot = tick_value / tick_size  # value of 1 full price unit per
lot
    lot_size = risk_amount / (atr_stop_distance * pip_value_per_lot)
    lot_size = round(lot_size, 2)
    lot_size = max(0.01, min(lot_size, profile['max_lot']))
    return lot_size
```

Fix 4 — Add Spread-to-ATR Ratio Filter

Add to order_router.py:

```
def check_spread_atr_ratio(self, symbol: str, atr14_price: float) -> tuple[bool,
str]:
    """Block entry if spread exceeds 15% of ATR — PRD Volume II Section 21.1."""
    tick = mt5.symbol_info_tick(self.mt5.normalize_symbol(symbol))
    if tick is None:
        return False, 'Cannot fetch tick data for spread check'
    symbol_info = mt5.symbol_info(self.mt5.normalize_symbol(symbol))
    spread_price = (tick.ask - tick.bid)
    if atr14_price <= 0:
        return False, 'ATR is zero — cannot compute spread ratio'
    ratio = spread_price / atr14_price
    if ratio > 0.15:
        return False, f'Spread/ATR ratio {ratio:.3f} exceeds 0.15 limit'
    return True, f'Spread/ATR ratio {ratio:.3f} acceptable'
```

Fix 5 — Per-Pair Daily Loss Tracking

Add to database.py and manager.py:

```
# In database.py:
def get_daily_realised_pnl(self, symbol: str) -> float:
    """Sum P&L of all closed trades for symbol since UTC midnight today."""
    today_midnight =
datetime.now(timezone.utc).replace(hour=0,minute=0,second=0,microsecond=0).isoforma
t()
    with self.get_connection() as conn:
        row = conn.execute(
            'SELECT COALESCE(SUM(profit), 0) FROM trades WHERE symbol=? AND
close_time>=? AND status="CLOSED"',
            (symbol, today_midnight)
        ).fetchone()
    return row[0] if row else 0.0

# In manager.py check_trade_allowed():
daily_pnl = self.db.get_daily_realised_pnl(symbol)
daily_loss_pct = abs(min(0, daily_pnl)) / account_balance * 100
max_daily = self.INSTRUMENT_PROFILES[symbol].get('max_daily_loss', 3.0)
if daily_loss_pct >= max_daily:
    return False, f'{symbol} daily loss {daily_loss_pct:.2f}% exceeds {max_daily}%
limit'
```

Fix 6 — 4-Attempt Retry Logic with Exact Millisecond Delays

```
RETRY_CONFIG = [
    {'delay_ms': 0,    'slippage_pips': 0.3},
    {'delay_ms': 500,  'slippage_pips': 0.5},
    {'delay_ms': 1500, 'slippage_pips': 0.8},
    {'delay_ms': 4000, 'slippage_pips': 1.2},
]

def place_order(self, symbol, direction, lots, sl=None, tp=None, comment=''):
    for attempt_idx, cfg in enumerate(RETRY_CONFIG):
        if cfg['delay_ms'] > 0:
            time.sleep(cfg['delay_ms'] / 1000)
        deviation = int(cfg['slippage_pips'] / mt5.symbol_info(symbol).point)
        # ... build request dict with deviation ...
        result = mt5.order_send(request)
        if result and result.retcode == mt5.TRADE_RETCODE_DONE:
            return True, 'Order filled', result.order
        self.logger.warning('order_retry', attempt=attempt_idx+1,
retcode=result.retcode if result else 'None')
    # All 4 attempts failed
    self.db.log_missed_signal(symbol, direction, lots, 'Max retries exceeded')
    self.telegram.send_alert('WARNING', f'MISSED SIGNAL: {symbol} {direction} after
4 attempts')
    return False, 'Order failed after 4 attempts', None
```

Fix 7 & 8 — Time-Based Exits and Weekend Policy

These are implemented in the new time_exit_manager.py module detailed in Part 5 below. Wire it in the main loop immediately after fixing Gaps 1-6.

PART 5 — NEW MODULE SPECIFICATIONS (VOLUME II)

Build each module below as a standalone Python class in the specified file path. Each class must be importable and testable independently. All classes receive config dict and logger in __init__().

Module 1: src/risk/portfolio_heat_monitor.py

Class: PortfolioHeatMonitor. Computes and enforces portfolio heat — the sum of potential loss across all open positions if every stop-loss is hit simultaneously.

```
class PortfolioHeatMonitor:
    HEAT_LEVELS = {
        'COLD':     {'max_pct': 2.0, 'rule': 'All pairs trade freely'},
        'WARM':     {'max_pct': 3.0, 'rule': 'New entries only if signal score >=
80'},
        'HOT':      {'max_pct': 4.0, 'rule': 'No new positions'},
        'CRITICAL': {'max_pct': 999, 'rule': 'No new positions + partial close
lowest conviction'},
    }

    def calculate_heat(self, positions: list, account_equity: float) ->
tuple[float, str]:
        """
        Compute total portfolio heat as % of equity.
        For each position: risk = abs(current_price - stop_loss) * lots *
pip_value_per_lot
        Sum all risks, divide by equity.
        Returns (heat_pct, heat_level_name).
        """
```

```
    def can_open_new_position(self, current_heat: float, signal_score: float) ->
tuple[bool, str]:
        """Returns (allowed, reason) based on current heat level and signal
score."""

    def get_partial_close_candidate(self, positions: list, signal_scores: dict) ->
int | None:
        """
        When heat is CRITICAL, return the ticket of the lowest-conviction
        open trade (lowest signal score at entry). Returns None if no scores
recorded.
        """
```

Module 2: src/risk/volatility_scaler.py

Class: VolatilityScaler. Computes a portfolio-level scalar (0.40 to 1.00) applied to all position sizes based on current realized volatility vs 60-day average.

```
class VolatilityScaler:
    VRS_TABLE = [
        {'max_ratio': 0.8,  'scalar': 1.00, 'label': 'CALM'},
        {'max_ratio': 1.2,  'scalar': 1.00, 'label': 'NORMAL'},
        {'max_ratio': 1.5,  'scalar': 0.70, 'label': 'ELEVATED'},
        {'max_ratio': 2.0,  'scalar': 0.50, 'label': 'HIGH'},
        {'max_ratio': 9999, 'scalar': 0.40, 'label': 'CRISIS'},
    ]

    def compute_realized_vol_ratio(self, symbol: str, df_d1: pd.DataFrame) ->
float:
        """5-day sum of daily ATR / 60-day average daily ATR for the symbol."""

    def detect_correlation_spike(self, all_symbols_ratios: dict[str, float]) ->
bool:
        """Return True if 3+ symbols simultaneously show vol ratio > 1.3."""

    def get_scalar(self, vol_ratio: float, correlation_spike: bool) -> tuple[float,
str]:
        """Return (scalar_value, label). If correlation_spike, cap scalar at
0.40."""

    def is_xau_defensive_mode(self, xau_df_h4: pd.DataFrame) -> bool:
        """True if XAU/USD ATR(14) on H4 exceeds 2.5x its 20-day ATR average."""

    def get_xau_defensive_sl_multiplier(self) -> float:
        """Return 0.70 — trailing stop distance reduced 30% in Defensive Mode."""
```

Module 3: src/risk/signal_scorer.py

Class: SignalScorer. Evaluates every entry candidate on a 0-100 scale across 5 weighted factor groups. Replaces binary gating. COT data plugs in as a bonus/penalty.

```
class SignalScorer:
    # Factor weights must sum to 100
    FACTORS = {
        'trend_alignment':   {'weight': 30, 'desc': 'D1+H4+H1 EMA alignment'},
        'momentum':          {'weight': 25, 'desc': 'RSI zone + MACD direction'},
        'regime_quality':    {'weight': 20, 'desc': 'ADX strength or BB squeeze'},
        'session_timing':    {'weight': 15, 'desc': 'Optimal vs secondary window'},
        'execution_quality': {'weight': 10, 'desc': 'Spread percentile for
window'},
```

```
    }
    COT_BONUS = 10    # Added when COT aligns with trade direction
    COT_PENALTY = 15  # Subtracted when COT is a contraindication

    GRADES = {
        (0, 59):   {'grade': 'REJECT',         'modifier': 0.0},
        (60, 74):  {'grade': 'MARGINAL',        'modifier': 0.60},
        (75, 84):  {'grade': 'STANDARD',        'modifier': 1.00},
        (85, 100): {'grade': 'HIGH_CONVICTION', 'modifier': 1.25},
    }

    def score(self, symbol: str, direction: str, indicators: dict,
              session_window: str, spread_percentile: float,
              cot_index: float | None) -> dict:
        """
        Returns dict with:
          total_score, grade, position_modifier,
          factor_breakdown (dict of each component score),
          cot_adjustment
        """

    def _score_trend_alignment(self, symbol, direction, indicators) -> float:
        """30 pts max. D1+H4+H1 all aligned = 30. Two = 20. One = 10. None = 0."""

    def _score_momentum(self, indicators, direction) -> float:
        """25 pts max. RSI in ideal zone + MACD confirming = 25. One = 12. Neither
= 0."""

    def _score_regime(self, indicators) -> float:
        """20 pts max. ADX > 30 or BB squeeze extreme = 20. ADX 25-30 = 12. Below =
0."""

    def _score_session(self, symbol, session_window) -> float:
        """15 pts max. Optimal session = 15. Secondary = 8. Outside preferred =
0."""

    def _score_execution(self, spread_percentile: float) -> float:
        """10 pts max. < 40th pct = 10. 40-70th = 6. 70-80th = 3. Above = 0."""

    def _apply_cot(self, score: float, symbol: str, direction: str,
                   cot_index: float | None) -> tuple[float, float]:
        """Apply COT bonus (+10) or penalty (-15). Returns (adjusted_score,
adjustment_amount)."""
```

Module 4: src/risk/kelly_sizer.py

Class: KellySizer. Computes fractional Kelly risk percentage based on rolling live trade performance. Only activates after minimum trade count threshold per instrument.

```
class KellySizer:
    INSTRUMENT_PARAMS = {
        'EURUSD': {'lookback':50, 'min_floor':0.5, 'max_cap':2.0,
'activation_trades':30},
        'GBPUSD': {'lookback':50, 'min_floor':0.4, 'max_cap':1.8,
'activation_trades':30},
        'USDJPY': {'lookback':50, 'min_floor':0.5, 'max_cap':2.0,
'activation_trades':30},
        'USDCHF': {'lookback':40, 'min_floor':0.3, 'max_cap':1.5,
'activation_trades':25},
        'USDCAD': {'lookback':50, 'min_floor':0.4, 'max_cap':1.8,
'activation_trades':30},
```

```
        'XAUUSD': {'lookback':30, 'min_floor':0.25,'max_cap':1.0,
'activation_trades':20},
    }
    KELLY_FRACTION = 0.25  # Use 25% of full Kelly — conservative fractional Kelly

    def get_effective_risk_pct(self, symbol: str, db) -> float:
        """
        Compute effective risk % for next trade on symbol.
        If live trade count < activation_trades: return Vol I fixed risk from
INSTRUMENT_PROFILES.
        Otherwise: compute Kelly from last N trades, apply 25% fraction, clamp to
floor/cap.
        """

    def _compute_kelly(self, win_rate: float, avg_win_r: float, avg_loss_r: float)
-> float:
        """
        Kelly formula: f* = (W * R - (1 - W)) / R
        where W=win_rate, R=avg_win/avg_loss (reward:risk ratio).
        Returns 0.0 if result is negative (negative edge — do not trade).
        """

    def recalculate_and_store(self, symbol: str, db) -> dict:
        """Run weekly recalculation and write result to kelly_history table."""

    def apply_cold_streak_ladder(self, symbol: str, base_risk_pct: float, db) ->
float:
        """
        Cold streak ladder from PRD Vol II Section 12.3:
        3 consecutive losses: * 0.75 for 5 trades
        5 consecutive losses: * 0.50 for 8 trades
        7+ consecutive losses: pair suspended — return 0.0
        """
```

Module 5: src/risk/fill_analyzer.py

Class: FillAnalyzer. Records and analyzes execution quality for every completed order. Computes the weekly Broker Quality Score (BQS).

```
class FillAnalyzer:
    BQS_WEIGHTS = {
        'shortfall_score':  0.35,
        'requote_score':    0.30,
        'latency_score':    0.20,
        'spread_exp_score': 0.15,
    }
    BQS_ALERT_THRESHOLD = 65  # Alert if BQS < 65 for 2 consecutive weeks

    def record_fill(self, symbol: str, ticket: int, expected_price: float,
                    actual_price: float, spread_at_submit: float,
                    spread_at_fill: float, latency_ms: float,
                    was_requoted: bool, was_partial: bool) -> None:
        """Write fill quality record to fill_quality table."""

    def compute_bqs(self, days: int = 7) -> dict:
        """
        Compute Broker Quality Score for the last N days.
        Each component scored 0-100 then weighted.
        Returns dict with total_bqs and component breakdown.
        """

    def _score_shortfall(self, avg_shortfall_pips: float) -> float:
```

```
        """0 pips = 100 pts. Deduct 10 pts per 0.1 pip of average shortfall."""

    def _score_requote(self, requote_rate_pct: float) -> float:
        """0% = 100. > 3% = 0. Linear interpolation."""

    def _score_latency(self, avg_latency_ms: float) -> float:
        """< 100ms = 100. > 350ms = 0. Linear interpolation."""

    def _score_spread_expansion(self, avg_expansion_pct: float) -> float:
        """0% expansion = 100. > 30% expansion = 0. Linear interpolation."""

    def get_spread_percentile(self, symbol: str, session_window: str) -> float:
        """
        Return the current spread's percentile within the 20-day rolling
        distribution for the given symbol and 30-minute session window.
        Percentile is used by SignalScorer for execution_quality scoring.
        """
```

Module 6: src/risk/regime_drift_detector.py

Class: RegimeDriftDetector. Runs every Friday at 21:00 UTC. Compares rolling live performance against backtest baseline for each instrument and triggers automated responses.

```
class RegimeDriftDetector:
    STATUS_RULES = {
        'GREEN':    {'min_ratio': 0.70, 'action': None},
        'AMBER':    {'min_ratio': 0.50, 'action': 'reduce_size_60pct'},
        'RED':      {'min_ratio': 0.00, 'action': 'suspend_pair'},
        'CRITICAL': {'min_ratio': 0.00, 'action': 'immediate_suspend'},
    }

    def run_weekly_check(self, db) -> dict[str, dict]:
        """
        For each symbol, compute rolling 30-trade profit factor.
        Compare against baseline from rdd_baselines in config.
        Determine status. Execute automated response. Log to rdd_status table.
        Returns dict: {symbol: {status, live_pf, baseline_pf, ratio, action}}
        """

    def _get_rolling_profit_factor(self, symbol: str, last_n: int, db) -> float:
        """Sum of winning trade profits / sum of losing trade losses for last N
closed trades."""

    def _execute_response(self, symbol: str, status: str, config) -> str:
        """
        GREEN: no action.
        AMBER: set size_modifier = 0.60 in config for symbol.
        RED: set symbol_suspended = True. Trigger re-optimization pipeline
notification.
        CRITICAL: set symbol_suspended = True immediately. Require manual reset.
        Returns description of action taken.
        """

    def check_critical_override(self, symbol: str, db) -> bool:
        """
        Critical override: current week loss > 2x worst backtest week for this
symbol.
        If True, suspend immediately regardless of profit factor.
        """
```

Module 7: src/analysis/performance_analyzer.py

Class: PerformanceAnalyzer. Self-learning feedback loop. Runs every Friday at 21:30 UTC. Clusters wins vs losses across 6 dimensions and produces governed parameter micro-corrections.

```
class PerformanceAnalyzer:
    ANALYSIS_DIMENSIONS = [
        'session_window',    # UTC 2-hour buckets
        'vrs_level',         # Volatility scalar at entry
        'adx_bucket',        # <20, 20-25, 25-30, 30-40, >40
        'signal_score_band', # 60-69, 70-79, 80-89, 90-100
        'day_of_week',       # 0=Monday ... 4=Friday
        'trade_duration_h',  # <4, 4-12, 12-24, 24-48, >48
    ]
    MAX_PARAM_CHANGE_PCT = 0.15  # No single parameter may shift > 15% per week
    MIN_TRADES_TO_ACT = 15       # Minimum trades per symbol for actionable results

    def run_weekly_analysis(self, db) -> dict:
        """
        Main entry point. For each symbol with >= MIN_TRADES_TO_ACT trades in last
30 days:
          1. Load trades with their signal scores and regime context.
          2. Cluster by each dimension — compute win_rate per bucket.
          3. Identify underperforming buckets (win_rate < overall_win_rate - 10%).
          4. Generate parameter adjustment recommendations.
          5. Apply governance gate. Write approved changes to
pending_config_changes table.
          6. Write weekly_report record to DB.
        Returns summary dict.
        """

    def _cluster_by_dimension(self, trades: list, dimension: str) -> dict:
        """Group trades by dimension bucket. Return {bucket: {wins, losses,
win_rate}}."""

    def _apply_governance_gate(self, recommendations: list) -> list:
        """
        Governance rules:
        - Max 1 parameter category updated per symbol per week.
        - Max 15% change from current value.
        - Changes only applied Sunday 22:00 UTC.
        - Every change logged with statistical justification.
        Returns filtered list of approved changes.
        """
```

Module 8: src/data/cot_parser.py

Class: COTParser. Downloads CFTC COT report weekly, computes COT Index per instrument, writes to cot_data table. COT Index is used by SignalScorer.

```
class COTParser:
    CFTC_URL = 'https://www.cftc.gov/files/dea/history/fut_disagg_txt_{year}.zip'
    CONTRACT_MAP = {
        'EURUSD': 'EURO FX',
        'GBPUSD': 'BRITISH POUND',
        'USDJPY': 'JAPANESE YEN',
        'USDCHF': 'SWISS FRANC',
        'USDCAD': 'CANADIAN DOLLAR',
        'XAUUSD': 'GOLD',
    }
    # COT contraindication thresholds per symbol (from PRD Vol II Section 18.3)
    CONTRAINDICATION = {
        'EURUSD': {'long_extreme': 90, 'short_extreme': 10},
```

```
        'GBPUSD': {'long_extreme': 90, 'short_extreme': 10},
        'USDJPY': {'long_extreme': 90, 'short_extreme': 10},
        'XAUUSD': {'long_extreme': 95, 'short_extreme': 5},
    }

    def fetch_and_update(self, db) -> dict[str, dict]:
        """Download latest COT CSV, parse, compute COT Index, write to cot_data
table."""

    def compute_cot_index(self, symbol: str, net_position: float, db) -> float:
        """
        COT Index = (Current - 3Y Min) / (3Y Max - 3Y Min) * 100
        Fetch 3Y min/max from cot_data table historical records.
        Returns 0-100 float.
        """

    def get_cot_signal(self, symbol: str, direction: str, db) -> tuple[str, float]:
        """
        Returns (signal_type, score_adjustment) where signal_type is:
        'CONFIRM' (+10), 'NEUTRAL' (0), or 'CONTRAINDICATE' (-15).
        Uses COT Index from latest cot_data record for symbol.
        """
```

Module 9: src/execution/time_exit_manager.py

Class: TimeExitManager. Enforces maximum hold times, forward progress rule, and weekend position policy on every bar iteration.

```
class TimeExitManager:
    MAX_HOLD_HOURS = {
        'EURUSD': {'trend': 36, 'breakout': 36},
        'GBPUSD': {'momentum': 18, 'fibonacci': 18},
        'USDJPY': {'carry': 48},
        'USDCHF': {'mean_reversion': 12},
        'USDCAD': {'oil_trend': 36},
        'XAUUSD': {'trend': 72, 'breakout': 72},
    }
    FORWARD_PROGRESS_ATR_THRESHOLD = 0.3  # < 0.3x ATR movement = no progress
    FORWARD_PROGRESS_HOURS_WARN = 6       # Hours before breakeven move
    FORWARD_PROGRESS_HOURS_EXIT = 10      # Hours before market exit
    XAU_FORWARD_HOURS_WARN = 12
    XAU_FORWARD_HOURS_EXIT = 18

    def check_all_positions(self, positions: list, atr_map: dict, mt5,
order_router) -> list[str]:
        """
        Called on every H1 bar close. For each open position:
          1. Check max hold time — exit if exceeded.
          2. Check forward progress rule — move to breakeven or exit.
          3. If Friday >= 20:00 UTC, apply weekend policy.
        Returns list of actions taken (for logging).
        """

    def _check_max_hold(self, position: dict, now_utc) -> bool:
        """Return True if position has exceeded its max hold time."""

    def _check_forward_progress(self, position: dict, atr14: float, now_utc,
order_router) -> str:
        """
        If price has not moved > 0.3x ATR toward TP since entry:
          After WARN hours: move SL to breakeven.
          After EXIT hours: close position at market.
```

```
        Returns action string.
        """

    def apply_weekend_policy(self, positions: list, atr_map: dict, order_router) ->
list[str]:
        """
        Called on Friday >= 20:00 UTC.
        Close all losing positions.
        Tighten trailing stops on profitable positions to 0.5x ATR.
        Only hold through weekend if profit >= 1.0x ATR AND trailing stop active.
        """
```

Module 10: src/execution/calendar_filter.py

Class: CalendarFilter. Enforces day-of-week rules, month-end/quarter-end size reductions, and seasonal adjustments.

```
class CalendarFilter:
    def get_size_modifier(self, now_utc: datetime) -> tuple[float, str]:
        """
        Returns (modifier, reason) to multiply against all position sizes.
        Checks in order: day-of-week rules, month-end, quarter-end, NFP week, FOMC
week.
        Returns the most restrictive (lowest) modifier that applies.
        """

    def is_new_entry_allowed(self, now_utc: datetime) -> tuple[bool, str]:
        """
        Returns (allowed, reason).
        Blocks: Monday before 10:00 UTC. Friday after 16:00 UTC.
        """

    def _is_month_end(self, now_utc: datetime, trading_days_from_end: int = 2) ->
bool:
        """True if within the last 2 trading days of the calendar month."""

    def _is_quarter_end(self, now_utc: datetime) -> bool:
        """True if within the last 5 trading days of March, June, September,
December."""

    def _is_nfp_week(self, now_utc: datetime) -> bool:
        """True if current week contains the first Friday of the month (NFP
day)."""
```

Module 11: src/data/swap_filter.py

Class: SwapFilter. Before entries within 2 hours of 22:00 UTC rollover, checks whether expected profit covers rollover cost by at least 2x.

```
class SwapFilter:
    ROLLOVER_UTC_HOUR = 22
    ROLLOVER_CHECK_WINDOW_HOURS = 2
    AFFECTED_INSTRUMENTS = ['GBPUSD', 'USDCHF']  # Short-duration strategies only
    MIN_PROFIT_TO_ROLLOVER_RATIO = 2.0

    def should_delay_entry(self, symbol: str, direction: str,
                           expected_tp_pips: float, mt5_connector) -> tuple[bool,
str]:
        """
        Returns (should_delay, reason).
```

```
        Fetch swap rate from MT5 symbol_info swap_long / swap_short.
        If expected_tp_profit < rollover_cost * 2.0 and within 2h of rollover:
delay.
        Only applies to AFFECTED_INSTRUMENTS.
        """

    def _get_swap_cost(self, symbol: str, direction: str, lots: float,
                       mt5_connector) -> float:
        """Fetch and return the rollover cost in account currency for 1 lot."""
```

PART 6 — INTEGRATION: WIRING EVERYTHING INTO main.py AND src/api/main.py

After all modules are built and tested individually, integrate them into the main execution loop and API layer.

6.1 Updated Main Execution Loop

The main scan loop in main.py must follow this sequence on every H1 bar close:

```
def run_bar_close_cycle(self):
    now_utc = datetime.now(timezone.utc)
    positions = self.mt5.get_positions()
    account = self.mt5.get_account()
    equity = account['equity']

    # Step 1: Compute portfolio-level state
    heat_pct, heat_level = self.heat_monitor.calculate_heat(positions, equity)
    vrs_scalar, vrs_label = self.vol_scaler.get_scalar(...)
    xau_defensive = self.vol_scaler.is_xau_defensive_mode(...)

    # Step 2: Monitor open positions
    self.monitor_trailing_stops()  # Fix 1
    self.time_exit_manager.check_all_positions(positions, atr_map, self.mt5,
self.order_router)

    # Step 3: Scan for new signals (if entry is allowed)
    if self.calendar_filter.is_new_entry_allowed(now_utc)[0]:
        for symbol in self.watchlist:
            if self.risk_manager.is_suspended(symbol): continue
            # ... fetch OHLC, compute regime, run strategy ...
            if signal:
                # Step 4: Score the signal
                cot_signal = self.cot_parser.get_cot_signal(symbol,
signal.direction, self.db)
                spread_pct = self.fill_analyzer.get_spread_percentile(symbol,
session_window)
                score_result = self.signal_scorer.score(symbol, signal.direction,
                    indicators, session_window, spread_pct, cot_index)
                if score_result['grade'] == 'REJECT': continue

                # Step 5: Compute position size
                kelly_risk = self.kelly_sizer.get_effective_risk_pct(symbol,
self.db)
                kelly_risk = self.kelly_sizer.apply_cold_streak_ladder(symbol,
kelly_risk, self.db)
                base_lots = self.risk_manager.calculate_position_size(symbol,
equity, atr14, direction)
                final_lots = base_lots * score_result['position_modifier'] *
vrs_scalar
                calendar_mod = self.calendar_filter.get_size_modifier(now_utc)[0]
```

```
                final_lots = final_lots * calendar_mod

                # Step 6: Pre-trade gate checks
                heat_ok, heat_reason = self.heat_monitor.can_open_new_position(
                    heat_pct, score_result['total_score'])
                spread_atr_ok, _ = self.order_router.check_spread_atr_ratio(symbol,
atr14)
                swap_ok, _ = self.swap_filter.should_delay_entry(symbol, direction,
tp_pips, self.mt5)
                corr_ok, _ = self.correlation_manager.can_open_position(symbol,
direction, kelly_risk)
                if not all([heat_ok, spread_atr_ok, not swap_ok, corr_ok]):
continue

                # Step 7: Execute
                submit_time = datetime.now(timezone.utc)
                success, msg, ticket = self.order_router.place_order(symbol,
direction, final_lots, sl, tp)
                if success:
                    fill_time = datetime.now(timezone.utc)
                    latency_ms = (fill_time - submit_time).total_seconds() * 1000
                    self.fill_analyzer.record_fill(symbol, ticket, entry_price,
actual_fill,
                        spread_at_submit, spread_at_fill, latency_ms, False, False)
                    self.db.log_signal_score(score_result, ticket)
```

6.2 New API Endpoints to Add

Add these endpoints to src/api/main.py:

```
GET  /api/v2/heat          # Current portfolio heat pct and level
GET  /api/v2/vrs           # Current VRS scalar, label, per-symbol vol ratios
GET  /api/v2/bqs           # Broker Quality Score + component breakdown
GET  /api/v2/kelly         # Current Kelly risk % per symbol + activation status
GET  /api/v2/rdd           # RDD status per symbol (GREEN/AMBER/RED/CRITICAL)
GET  /api/v2/cot           # Latest COT Index per symbol
GET  /api/v2/signal-scores # Last 50 signal scores with grades and outcomes
GET  /api/v2/weekly-report # Latest weekly performance report JSON
POST /api/v2/rdd/reset/<symbol>  # Manual reset of suspended symbol (with auth)
```

PART 7 — IMPLEMENTATION ORDER

Build in this exact sequence. Each session should end with all new tests passing. Do not move to the next session until current session tests are green.

Session	Deliverable	Success Criteria
1	Fix all 8 Critical Gaps from Section 2	All existing tests pass. Trailing stops fire in demo. Per-pair daily loss limits block trades correctly.
2	portfolio_heat_monitor.py + volatility_scaler.py	Heat level correctly blocks new entries when > 3%. VRS scalar reduces lot sizes during simulated high-vol conditions.
3	time_exit_manager.py + calendar_filter.py + swap_filter.py	Positions exit at max hold time. Friday 20:00 policy closes losing trades. Monday entries blocked before 10:00 UTC.

4	signal_scorer.py + fill_analyzer.py	Binary entry gates replaced by scorer. Trades scored correctly and REJECT signals blocked. Fill data written after each test order.
5	cot_parser.py + wire COT into signal_scorer	COT CSV parsed correctly. COT Index computed. CONTRAINDICATE signals deduct 15 points from score.
6	kelly_sizer.py + update manager.py	Fixed risk used until activation threshold. Kelly activates with correct math after threshold. Cold streak ladder cuts size after 3 losses.
7	regime_drift_detector.py + performance_analyzer.py	RDD correctly flags AMBER when live PF is 55% of baseline. Analyzer produces parameter recommendations within 15% change limit.
8	API endpoints + Telegram tiers + main.py full wiring	All /api/v2/ endpoints return correct data. CRITICAL alerts arrive with sound. Full 15-step pre-trade check executes in correct order.
9	Full integration test on MT5 demo account	Bot runs 48 hours uninterrupted on demo. All 6 pairs scanned. At least 3 trades executed. All logs clean. Weekly report generated.

PART 8 — TESTING REQUIREMENTS

Every module must have a corresponding test file. Tests must run without a live MT5 connection. Use unittest.mock.patch and pytest fixtures.

- tests/test_portfolio_heat_monitor.py — test COLD/WARM/HOT/CRITICAL transitions with mock positions
- tests/test_volatility_scaler.py — test VRS scalar values at each ratio threshold; test correlation spike detection with 3+ symbols
- tests/test_signal_scorer.py — test all 5 factor scoring functions independently; test COT bonus and penalty application; test that REJECT (< 60) blocks correctly
- tests/test_kelly_sizer.py — test Kelly formula with known win_rate/R inputs; test activation threshold gate; test cold streak ladder at 3, 5, and 7 consecutive losses
- tests/test_fill_analyzer.py — test BQS computation with mock fill records; test spread percentile calculation
- tests/test_time_exit_manager.py — test max hold time trigger; test forward progress rule at WARN and EXIT hours; test weekend policy
- tests/test_cot_parser.py — test COT Index computation with mock 3Y data; test CONTRAINDICATE detection at extreme levels
- tests/test_regime_drift_detector.py — test GREEN/AMBER/RED status transitions; test CRITICAL override trigger
- tests/test_calendar_filter.py — test Monday entry block; test Friday entry block after 16:00; test month-end modifier; test quarter-end modifier

PART 9 — FINAL SYSTEM CHECKLIST

Before declaring the bot production-ready, verify all items below:

	Requirement	How to Verify

[ ]	All 8 critical gaps from Section 2 are fixed	Run full test suite — all tests pass
[ ]	Position sizing formula matches PRD Volume I Section 5.2 exactly	Compute manually for one trade, compare to bot output
[ ]	All 11 new Volume II modules are present and importable	python -c 'from src.risk.signal_scorer import SignalScorer' (and each module)
[ ]	Pre-trade check executes all 15 steps in correct order (Section 6.1)	Add debug logging to execution loop; verify order in logs
[ ]	Kelly sizing activates only after minimum live trades threshold	Check kelly_history table — no entries before threshold count
[ ]	RDD runs every Friday at 21:00 UTC via APScheduler	Check APScheduler job list at startup
[ ]	EMERGENCY tier Telegram alert sends with sound for 8% drawdown	Simulate circuit breaker trigger in test environment
[ ]	No positions held through weekend with unrealised loss	Backtest weekend policy on historical data with losing trades
[ ]	All API v2 endpoints return correct JSON structure	Run integration tests against Flask test client
[ ]	48-hour demo run completes with no unhandled exceptions	Check errors.log — must be empty or contain only expected warnings

APEX FX TRADING BOT — Master AI Coding Prompt | Vol I + Vol II | April 2026 | CONFIDENTIAL

This prompt is authoritative. If any instruction here conflicts with older versions of the PRD, this prompt takes precedence.